

Contexto

Analiza los siguientes dos algoritmos:

```
// Algoritmo A
for (int i = 0; i < n; i++) {
    System.out.println(i);
}

// Algoritmo B
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.println(i + " " + j);
    }
}
```

- ¿Cuál crece más rápido cuando n aumenta?

El que crece mas es el algoritmo B, por que tiene un ciclo dentro de otro ciclo, por eso las operaciones aumenta más cuando (n) crece.

- ¿Cuál es más eficiente?

El más eficiente es el algoritmo A, ya que solo utiliza un ciclo (for), mientras que el algoritmo B solo usa dos ciclos anidados.

- Explica por qué.

Por ejemplo en el algoritmo A

```
for (int i = 0; i < n; i++) {
    System.out.println(i);
}
```

Este algoritmo repite el proceso (n) veces, su complejidad es $O(n)$, eso significa que el tiempo crece de manera lineal.

Ya para el algoritmo B

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.println(i + " " + j);
    }
}
```

En esta algoritmo el segundo (for) se ejecuta completo por cada vuelta del primero, entonces realiza aproximadamente $n \times n = n^2$, su complejidad $O(n^2)$, por eso tarda mas cuando n aumenta.